Day10 综合可视化与Dashboard

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 10 天（8 课时）
> 主题：综合可视化与Dashboard，把所有分析结果整合为统一的可视化展示

一、本日目标

1. 理解综合可视化的设计原则和Dashboard布局；
2. 掌握matplotlib高级技巧（子图、布局、样式、中文显示）；
3. 掌握出行驻留结果可视化（标签分布、速度分布、空间分布）；
4. 掌握出行目的与方式可视化；
5. 掌握实时热力可视化（热力图、时间趋势）；
6. 掌握3D轨迹可视化；
7. 完整实现综合可视化模块（visualize.py）；
8. 可视化优化与答辩准备。

二、知识要点

- 综合可视化把所有分析结果整合为6张图：标签饼图、目的柱状图、方式柱状图、小时折线图、热力图、3D轨迹图；
- matplotlib高级技巧：subplots子图、gridspec布局、rcParams样式、colorbar；
- 中文显示：SimHei字体，axes.unicode_minus=False；
- 3D可视化：mpl_toolkits.mplot3d，经纬度-时间三维轨迹；
- 输出：6张PNG图片，分辨率150dpi。

课时1：综合可视化总览与设计

1.1 课时目标

理解综合可视化的设计原则，掌握Dashboard布局设计方法。

1.2 知识点深度讲解

1.2.1 什么是综合可视化

综合可视化是把所有分析结果（出行驻留、出行目的、出行方式、实时热力）整合为统一的可视化展示，方便用户一目了然地查看分析结果。

本项目的6张图：
1. **fig_stage3_pie**：出行/驻留标签分布饼图；
2. **fig_purpose_bar**：出行目的分布柱状图；
3. **fig_mode_bar**：出行方式分布柱状图；
4. **fig_hour_line**：24小时出行人次折线图；
5. **fig_heatmap**：商圈热力图；
6. **fig_3d_traj**：用户3D轨迹图。

1.2.2 可视化设计原则

1. **简洁性**：每张图只表达一个核心信息，避免信息过载；
2. **一致性**：所有图使用统一的颜色、字体、样式；
3. **可读性**：坐标轴标签、标题、图例清晰可读；
4. **美观性**：配色和谐，布局合理；
5. **可解释性**：每张图都有明确的标题和说明。

1.2.3 Dashboard布局设计

Dashboard是把多张图整合到一个页面的布局方式。常见布局：
1. **网格布局**：2×3网格，每张图占一个格子；
2. **侧边栏布局**：左侧导航，右侧内容；
3. **标签页布局**：每个分析模块一个标签页。

本项目用6张独立图片，也可以整合为一个Dashboard页面。

1.2.4 配色方案

本项目的配色方案：
- **驻留**：绿色（#2ca02c）
- **出行**：红色（#d62728）
- **居住**：深绿（#1f77b4）
- **购物**：蓝色（#ff7f0e）
- **旅游**：橙色（#9467bd）
- **其他**：灰色（#7f7f7f）
- **地铁**：红色（#d62728）

- **公交**：蓝色（#1f77b4）
- **小汽车**：橙色（#ff7f0e）

> 配色原则：同类使用相近色，不同类使用对比色，避免使用过于鲜艳的颜色。

1.2.5 中文显示问题

matplotlib默认不支持中文，需要配置：

```
plt.rcParams["font.sans-serif"] = ["SimHei"]  # 黑体
plt.rcParams["axes.unicode_minus"] = False     # 负号显示
```

> SimHei是Windows自带的中文字体，Linux/Mac可能需要安装或使用其他字体（如WenQuanYi Micro Hei、Arial Unicode MS）。

1.3 项目实现步骤

1. 设计6张图的内容和布局；
2. 配置matplotlib样式（中文、字体、颜色）；
3. 读取所有分析结果；
4. 逐张生成图片；
5. 保存为PNG文件。

1.4 完整代码（可视化配置与设计演示）

```
# ============================================================
# 课时1：综合可视化总览与设计
# ============================================================

import matplotlib.pyplot as plt
import numpy as np
import os

# matplotlib配置
plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 150
plt.rcParams["savefig.dpi"] = 150
plt.rcParams["savefig.bbox"] = "tight"

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

# 配色方案
COLORS = {
    "驻留": "#2ca02c",
    "出行": "#d62728",
    "居住": "#1f77b4",
    "购物": "#ff7f0e",
    "旅游": "#9467bd",
    "其他": "#7f7f7f",
    "地铁": "#d62728",
    "公交": "#1f77b4",
    "小汽车": "#ff7f0e",
}

print("=" * 60)
print("综合可视化设计")
print("=" * 60)
```

```
print(f"\n6张图设计:")
print(f"  1. fig_stage3_pie.png  - 出行/驻留标签分布饼图")
print(f"  2. fig_purpose_bar.png  - 出行目的分布柱状图")
print(f"  3. fig_mode_bar.png     - 出行方式分布柱状图")
print(f"  4. fig_hour_line.png    - 24小时出行人次折线图")
print(f"  5. fig_heatmap.png      - 商圈热力图")
print(f"  6. fig_3d_traj.png      - 用户3D轨迹图")

print(f"\n配色方案:")
for name, color in COLORS.items():
    print(f"  {name}: {color}")

print(f"\nmatplotlib配置:")
print(f"  字体: SimHei（中文）")
print(f"  分辨率: 150 dpi")
print(f"  保存格式: PNG")
print(f"  输出目录: {FIG_DIR}")

# 演示：中文显示测试
print(f"\n中文显示测试...")
fig, ax = plt.subplots(figsize=(6, 4))
ax.text(0.5, 0.5, "中文显示测试：交通时空大数据分析",
        ha="center", va="center", fontsize=16)
ax.set_title("中文显示测试")
ax.set_xlabel("X轴")
ax.set_ylabel("Y轴")
plt.savefig(os.path.join(FIG_DIR, "fig_chinese_test.png"))
plt.close()
print(f"  测试图已保存: fig_chinese_test.png")

# 演示：配色方案测试
print(f"\n配色方案测试...")
fig, ax = plt.subplots(figsize=(10, 4))
categories = list(COLORS.keys())
colors = list(COLORS.values())
ax.bar(range(len(categories)), [1]*len(categories), color=colors, edgecolor="black")
ax.set_xticks(range(len(categories)))
ax.set_xticklabels(categories, rotation=45, ha="right")
```

```
ax.set_title("配色方案测试")
ax.set_ylabel("高度")
plt.savefig(os.path.join(FIG_DIR, "fig_color_test.png"))
plt.close()
print(f"  测试图已保存: fig_color_test.png")


print(f"\n可视化配置完成！")
```

1.5 运行结果与解读

运行后展示：
- 6张图的设计说明；
- 配色方案；
- matplotlib配置；
- 中文显示测试图；
- 配色方案测试图。

> 关键理解：
> 1. 综合可视化把所有分析结果整合为6张图；
> 2. 配色方案统一，同类相近色，不同类对比色；
> 3. 中文显示需要配置SimHei字体；
> 4. 所有图保存为PNG，分辨率150dpi。

1.6 当堂练习

1. 查看中文显示测试图，确认中文是否正常显示；
2. 思考：为什么需要统一的配色方案？（提示：保持视觉一致性，方便对比）
3. 尝试修改配色方案，使用其他颜色组合。

1.7 课时小结

- 综合可视化：6张图（标签饼图、目的柱状图、方式柱状图、小时折线图、热力图、3D轨迹图）；
- 设计原则：简洁、一致、可读、美观、可解释；
- 配色方案：统一颜色，同类相近色，不同类对比色；
- 中文显示：SimHei字体，axes.unicode_minus=False；
- 输出：PNG图片，150dpi。

课时2：matplotlib高级技巧

2.1 课时目标

掌握matplotlib高级技巧：子图、布局、样式、图例、注释。

2.2 知识点深度讲解

2.2.1 子图（subplots）

matplotlib的subplots可以创建多个子图：

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
# axes是一个2×2的数组
axes[0, 0].plot(...)
axes[0, 1].bar(...)
axes[1, 0].scatter(...)
axes[1, 1].pie(...)
```

2.2.2 高级布局（gridspec）

gridspec可以创建更灵活的布局：

```
import matplotlib.gridspec as gridspec
fig = plt.figure(figsize=(12, 8))
gs = gridspec.GridSpec(2, 2, figure=fig)
ax1 = fig.add_subplot(gs[0, :])  # 第一行占两列
ax2 = fig.add_subplot(gs[1, 0])  # 第二行第一列
ax3 = fig.add_subplot(gs[1, 1])  # 第二行第二列
```

2.2.3 样式配置（rcParams）

全局样式配置：

```
plt.rcParams["font.size"] = 12
plt.rcParams["axes.labelsize"] = 12
plt.rcParams["axes.titlesize"] = 14
plt.rcParams["xtick.labelsize"] = 10
plt.rcParams["ytick.labelsize"] = 10
plt.rcParams["legend.fontsize"] = 10
```

2.2.4 图例（legend）

图例的位置和样式：

```
ax.legend(loc="upper right", title="图例", fontsize=10)
# loc可选：upper right, upper left, lower right, lower left, best, center
```

2.2.5 注释（annotate）

在图上添加注释和箭头：

```
ax.annotate("这是峰值", xy=(x_peak, y_peak),
            xytext=(x_peak+2, y_peak*0.8),
            arrowprops=dict(arrowstyle="->", color="red"),
            color="red", fontsize=12)
```

2.2.6 双Y轴（twinx）

两个Y轴共享X轴：

```
fig, ax1 = plt.subplots()
ax1.plot(x, y1, color="blue")
ax1.set_ylabel("Y1", color="blue")
ax2 = ax1.twinx()
ax2.plot(x, y2, color="red")
ax2.set_ylabel("Y2", color="red")
```

2.2.7 颜色条（colorbar）

热力图或散点图的颜色条：

```
im = ax.imshow(data, cmap="YlOrRd")
plt.colorbar(im, ax=ax, label="数值")
```

2.3 项目实现步骤

1. 创建2×2子图布局；
2. 在每个子图中绘制不同类型的图；
3. 添加标题、标签、图例、注释；
4. 调整布局和样式；

5. 保存图片。

2.4 完整代码（matplotlib高级技巧）

```
# ============================================================
# 课时2：matplotlib高级技巧
# ============================================================

import matplotlib.pyplot as plt
import numpy as np
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

# 生成模拟数据
np.random.seed(42)
x = np.linspace(0, 10, 100)
y1 = np.sin(x) + np.random.normal(0, 0.1, 100)
y2 = np.cos(x) + np.random.normal(0, 0.1, 100)
categories = ["A", "B", "C", "D", "E"]
values = [23, 45, 56, 78, 34]
scatter_x = np.random.rand(50) * 10
scatter_y = np.random.rand(50) * 10
scatter_size = np.random.rand(50) * 100 + 20

# 图1：2×2子图
print("生成图1：2×2子图")
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# 子图1：折线图
axes[0, 0].plot(x, y1, label="sin", color="blue", linewidth=2)
axes[0, 0].plot(x, y2, label="cos", color="red", linewidth=2)
axes[0, 0].set_title("折线图")
axes[0, 0].set_xlabel("X")
```

```
axes[0, 0].set_ylabel("Y")
axes[0, 0].legend(loc="upper right")
axes[0, 0].grid(alpha=0.3)

# 子图2：柱状图
bars = axes[0, 1].bar(categories, values, color="steelblue", edgecolor="black")
axes[0, 1].set_title("柱状图")
axes[0, 1].set_xlabel("类别")
axes[0, 1].set_ylabel("数值")
for bar, val in zip(bars, values):
    axes[0, 1].text(bar.get_x() + bar.get_width()/2, val + 1, str(val), ha="center")

# 子图3：散点图
sc = axes[1, 0].scatter(scatter_x, scatter_y, s=scatter_size, c=scatter_size,
                        cmap="viridis", alpha=0.7, edgecolors="black")
axes[1, 0].set_title("散点图（大小+颜色映射）")
axes[1, 0].set_xlabel("X")
axes[1, 0].set_ylabel("Y")
plt.colorbar(sc, ax=axes[1, 0], label="大小")

# 子图4：饼图
axes[1, 1].pie(values, labels=categories, autopct="%1.1f%%",
               colors=["#ff9999", "#66b3ff", "#99ff99", "#ffcc99", "#c2c2f0"],
               startangle=90)
axes[1, 1].set_title("饼图")

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_subplots_demo.png"))
plt.close()

# 图2：gridspec高级布局
print("生成图2：gridspec高级布局")
import matplotlib.gridspec as gridspec
fig = plt.figure(figsize=(12, 8))
gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.3)

# 第一行占两列
ax1 = fig.add_subplot(gs[0, :])
```

```
ax1.plot(x, y1, color="blue", linewidth=2)
ax1.set_title("第一行（占两列）")
ax1.set_xlabel("X")
ax1.set_ylabel("Y")
ax1.grid(alpha=0.3)
# 注释峰值
peak_idx = np.argmax(y1)
ax1.annotate(f"峰值: {y1[peak_idx]:.2f}",
             xy=(x[peak_idx], y1[peak_idx]),
             xytext=(x[peak_idx]+1, y1[peak_idx]+0.5),
             arrowprops=dict(arrowstyle="->", color="red"),
             color="red")

# 第二行第一列
ax2 = fig.add_subplot(gs[1, 0])
ax2.bar(categories, values, color="steelblue", edgecolor="black")
ax2.set_title("第二行第一列")
ax2.set_xlabel("类别")
ax2.set_ylabel("数值")

# 第二行第二列（双Y轴）
ax3 = fig.add_subplot(gs[1, 1])
ax3.plot(x, y1, color="blue", label="sin")
ax3.set_ylabel("sin", color="blue")
ax3.tick_params(axis="y", labelcolor="blue")
ax3_twin = ax3.twinx()
ax3_twin.plot(x, y2 * 10, color="red", label="cos×10")
ax3_twin.set_ylabel("cos×10", color="red")
ax3_twin.tick_params(axis="y", labelcolor="red")
ax3.set_title("双Y轴")
ax3.set_xlabel("X")

plt.savefig(os.path.join(FIG_DIR, "fig_gridspec_demo.png"))
plt.close()

# 图3：样式配置
print("生成图3：样式配置")
plt.rcParams["font.size"] = 14
```

```
plt.rcParams["axes.labelsize"] = 14
plt.rcParams["axes.titlesize"] = 16
plt.rcParams["xtick.labelsize"] = 12
plt.rcParams["ytick.labelsize"] = 12
plt.rcParams["legend.fontsize"] = 12
plt.rcParams["lines.linewidth"] = 3
plt.rcParams["lines.markersize"] = 8

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y1, marker="o", markevery=10, label="sin")
ax.plot(x, y2, marker="s", markevery=10, label="cos")
ax.set_title("样式配置演示（大字体+粗线+标记）")
ax.set_xlabel("X轴")
ax.set_ylabel("Y轴")
ax.legend(loc="best")
ax.grid(alpha=0.3, linestyle="--")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_style_demo.png"))
plt.close()

# 恢复默认样式
plt.rcParams.update(plt.rcParamsDefault)
plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

print(f"\n图片保存在: {FIG_DIR}")
print(f"  fig_subplots_demo.png")
print(f"  fig_gridspec_demo.png")
print(f"  fig_style_demo.png")
```

2.5 运行结果与解读

生成3张演示图：2×2子图、gridspec高级布局、样式配置。

> 关键理解：
> 1. subplots创建规则的2×2子图；
> 2. gridspec创建灵活布局（如第一行占两列）；
> 3. rcParams全局配置字体、大小、线宽；
> 4. annotate添加注释和箭头；

5. twinx创建双Y轴；
6. colorbar添加颜色条。

2.6 当堂练习

1. 创建一个3×1的子图布局，分别绘制折线图、柱状图、饼图；
2. 在折线图上添加峰值和谷值的注释；
3. 尝试修改rcParams，使用不同的字体大小和颜色。

2.7 课时小结

- 子图：subplots创建规则布局，gridspec创建灵活布局；
- 样式：rcParams全局配置字体、大小、线宽；
- 图例：legend指定位置和标题；
- 注释：annotate添加文字和箭头；
- 双Y轴：twinx共享X轴；
- 颜色条：colorbar显示数值映射。

课时3：出行驻留结果可视化

3.1 课时目标

可视化出行驻留识别结果：标签分布饼图、速度分布、空间分布。

3.2 完整代码（出行驻留结果可视化）

```
# ============================================================
# 课时3：出行驻留结果可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

# 读取阶段3结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
print(f"阶段3结果：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 图1：标签分布饼图
print("生成图1：标签分布饼图")
label_counts = df["label"].value_counts()
label_names = {0: "驻留", 1: "出行"}
labels = [label_names.get(l, str(l)) for l in label_counts.index]
colors = ["#2ca02c", "#d62728"]

fig, ax = plt.subplots(figsize=(8, 6))
wedges, texts, autotexts = ax.pie(label_counts.values, labels=labels,
                                  autopct="%1.1f%%", colors=colors,
                                  startangle=90, textprops={"fontsize": 14})
for autotext in autotexts:
    autotext.set_color("white")
    autotext.set_fontsize(14)
```

```
    autotext.set_fontweight("bold")
ax.set_title("出行/驻留标签分布", fontsize=16, fontweight="bold")
# 添加数量标注
for i, (wedge, count) in enumerate(zip(wedges, label_counts.values)):
    angle = (wedge.theta2 + wedge.theta1) / 2
    x = 1.3 * np.cos(np.radians(angle))
    y = 1.3 * np.sin(np.radians(angle))
    ax.text(x, y, f"{count}点", ha="center", va="center", fontsize=12)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_pie.png"), dpi=150, bbox_inches="tight")
plt.close()

# 图2：24小时出行人次折线图
print("生成图2：24小时出行人次折线图")
df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
df["hour"] = df["time_dt"].dt.hour
travel_df = df[df["label"] == 1]
hourly_travel = travel_df.groupby("hour").size().reindex(range(24), fill_value=0)
stay_df = df[df["label"] == 0]
hourly_stay = stay_df.groupby("hour").size().reindex(range(24), fill_value=0)

fig, ax = plt.subplots(figsize=(12, 6))
ax.plot(hourly_travel.index, hourly_travel.values, marker="o", color="#d62728",
        linewidth=2, label="出行", markersize=6)
ax.plot(hourly_stay.index, hourly_stay.values, marker="s", color="#2ca02c",
        linewidth=2, label="驻留", markersize=6)
ax.fill_between(hourly_travel.index, hourly_travel.values, alpha=0.2, color="#d62728")
ax.set_xlabel("小时", fontsize=12)
ax.set_ylabel("人次", fontsize=12)
ax.set_title("24小时出行/驻留人次分布", fontsize=14, fontweight="bold")
ax.set_xticks(range(24))
ax.legend(fontsize=12)
ax.grid(alpha=0.3, linestyle="--")
# 标注早高峰和晚高峰
morning_peak = hourly_travel.iloc[6:10].idxmax()
evening_peak = hourly_travel.iloc[16:20].idxmax()
ax.annotate(f"早高峰: {morning_peak}时", xy=(morning_peak,
hourly_travel[morning_peak]),
```

```
            xytext=(morning_peak-3, hourly_travel[morning_peak]*1.1),
            arrowprops=dict(arrowstyle="->", color="red"), color="red", fontsize=11)
ax.annotate(f"晚高峰: {evening_peak}时", xy=(evening_peak,
hourly_travel[evening_peak]),
            xytext=(evening_peak+1, hourly_travel[evening_peak]*1.1),
            arrowprops=dict(arrowstyle="->", color="red"), color="red", fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_hour_line.png"), dpi=150, bbox_inches="tight")
plt.close()

# 图3: 空间分布散点图
print("生成图3: 空间分布散点图")
fig, ax = plt.subplots(figsize=(12, 8))
stay_sample = df[df["label"] == 0].sample(min(5000, len(df[df["label"]==0])),
random_state=42)
travel_sample = df[df["label"] == 1].sample(min(2000, len(df[df["label"]==1])),
random_state=42)
ax.scatter(stay_sample["lon"], stay_sample["lat"], c="#2ca02c", s=5, alpha=0.3,
label="驻留")
ax.scatter(travel_sample["lon"], travel_sample["lat"], c="#d62728", s=5, alpha=0.3,
label="出行")
ax.set_xlabel("经度", fontsize=12)
ax.set_ylabel("纬度", fontsize=12)
ax.set_title("出行/驻留空间分布（抽样）", fontsize=14, fontweight="bold")
ax.legend(markerscale=5, fontsize=12)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage3_spatial.png"), dpi=150,
bbox_inches="tight")
plt.close()

print(f"\n图片保存在: {FIG_DIR}")
print(f"  fig_stage3_pie.png")
print(f"  fig_hour_line.png")
print(f"  fig_stage3_spatial.png")

# 统计摘要
print(f"\n{'='*60}")
```

```
print("出行驻留统计摘要")
print(f"{'='*60}")
print(f"总点数: {len(df)}")
print(f"驻留点: {(df['label']==0).sum()} ({(df['label']==0).mean()*100:.1f}%)")
print(f"出行点: {(df['label']==1).sum()} ({(df['label']==1).mean()*100:.1f}%)")
print(f"出行高峰: {hourly_travel.idxmax()}时 ({hourly_travel.max()}人次)")
print(f"驻留高峰: {hourly_stay.idxmax()}时 ({hourly_stay.max()}人次)")
```

3.3 运行结果与解读

生成3张图：标签饼图、24小时折线图、空间分布散点图。

3.4 课时小结

- 标签饼图：驻留约87%，出行约13%；
- 24小时折线图：早高峰7-9点，晚高峰17-19点；
- 空间分布：驻留点聚集，出行点分散。

课时4：出行目的与方式可视化

4.1 课时目标

可视化出行目的和出行方式结果。

4.2 完整代码（出行目的与方式可视化）

```
# ============================================================
# 课时4：出行目的与方式可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")

# 图1：出行目的分布柱状图
print("生成图1：出行目的分布柱状图")
purpose_df = pd.read_csv(os.path.join(RESULT_DIR, "purpose_result.csv"),
                         dtype={"imsi": str, "start_time": str, "end_time": str})
purpose_counts = purpose_df["final_purpose"].value_counts()
purpose_order = ["居住", "购物", "旅游", "其他"]
purpose_counts = purpose_counts.reindex([p for p in purpose_order if p in
purpose_counts.index])
purpose_colors = ["#1f77b4", "#ff7f0e", "#9467bd", "#7f7f7f"]

fig, ax = plt.subplots(figsize=(10, 6))
bars = ax.bar(purpose_counts.index, purpose_counts.values, color=purpose_colors,
              edgecolor="black", linewidth=1.2)
ax.set_ylabel("驻留段数", fontsize=12)
ax.set_title("出行目的分布", fontsize=14, fontweight="bold")
for bar, val in zip(bars, purpose_counts.values):
    ax.text(bar.get_x() + bar.get_width()/2, val + 5,
            f"{val}\n({val/len(purpose_df)*100:.1f}%)",
            ha="center", va="bottom", fontsize=11, fontweight="bold")
ax.set_ylim(0, purpose_counts.max() * 1.2)
```

```
ax.grid(axis="y", alpha=0.3, linestyle="--")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_purpose_bar.png"), dpi=150, bbox_inches="tight")
plt.close()

# 图2：出行方式分布柱状图
print("生成图2：出行方式分布柱状图")
mode_df = pd.read_csv(os.path.join(RESULT_DIR, "mode_result.csv"),
                      dtype={"imsi": str, "start_time": str, "end_time": str})
mode_counts = mode_df["final_mode"].value_counts()
mode_order = ["地铁", "公交", "小汽车", "其他"]
mode_counts = mode_counts.reindex([m for m in mode_order if m in mode_counts.index])
mode_colors = ["#d62728", "#1f77b4", "#ff7f0e", "#7f7f7f"]

fig, ax = plt.subplots(figsize=(10, 6))
bars = ax.bar(mode_counts.index, mode_counts.values, color=mode_colors,
              edgecolor="black", linewidth=1.2)
ax.set_ylabel("出行段数", fontsize=12)
ax.set_title("出行方式分布", fontsize=14, fontweight="bold")
for bar, val in zip(bars, mode_counts.values):
    ax.text(bar.get_x() + bar.get_width()/2, val + 1,
            f"{val}\n({val/len(mode_df)*100:.1f}%)",
            ha="center", va="bottom", fontsize=11, fontweight="bold")
ax.set_ylim(0, mode_counts.max() * 1.2)
ax.grid(axis="y", alpha=0.3, linestyle="--")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_mode_bar.png"), dpi=150, bbox_inches="tight")
plt.close()

# 图3：出行目的 vs 出行方式（交叉热力图）
print("生成图3：出行目的 vs 出行方式")
# 注意：purpose和mode是不同的段，这里只做独立展示
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 目的饼图
axes[0].pie(purpose_counts.values, labels=purpose_counts.index, autopct="%1.1f%%",
            colors=purpose_colors, startangle=90, textprops={"fontsize": 11})
axes[0].set_title("出行目的分布（饼图）", fontsize=13, fontweight="bold")
```

```
# 方式饼图
axes[1].pie(mode_counts.values, labels=mode_counts.index, autopct="%1.1f%%",
            colors=mode_colors, startangle=90, textprops={"fontsize": 11})
axes[1].set_title("出行方式分布（饼图）", fontsize=13, fontweight="bold")

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_purpose_mode_pie.png"), dpi=150,
bbox_inches="tight")
plt.close()

print(f"\n图片保存在: {FIG_DIR}")
print(f"  fig_purpose_bar.png")
print(f"  fig_mode_bar.png")
print(f"  fig_purpose_mode_pie.png")

# 统计摘要
print(f"\n{'='*60}")
print("出行目的与方式统计摘要")
print(f"{'='*60}")
print(f"出行目的（{len(purpose_df)}段）:")
for purpose in purpose_order:
    if purpose in purpose_counts.index:
        print(f"  {purpose}: {purpose_counts[purpose]}
({purpose_counts[purpose]/len(purpose_df)*100:.1f}%)")
print(f"\n出行方式（{len(mode_df)}段）:")
for mode in mode_order:
    if mode in mode_counts.index:
        print(f"  {mode}: {mode_counts[mode]}
({mode_counts[mode]/len(mode_df)*100:.1f}%)")
```

4.3 运行结果与解读

生成3张图：目的柱状图、方式柱状图、目的+方式饼图。

4.4 课时小结

- 出行目的：其他最多，居住次之，购物/旅游较少；
- 出行方式：其他最多，小汽车次之，公交/地铁较少；

- 柱状图标注数量和比例，饼图展示占比。

课时5：实时热力可视化

5.1 课时目标

可视化实时热力结果：商圈热力图、时间趋势、空间分布。

5.2 完整代码（实时热力可视化）

```
# ============================================================
# 课时5：实时热力可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")

# 读取热力结果
df = pd.read_csv(os.path.join(RESULT_DIR, "heatmap_result.csv"))
df["time_dt"] = pd.to_datetime(df["time"])
print(f"热力结果：{len(df)} 行")

# 图1：商圈热力图（Top10商圈 × 24小时）
print("生成图1：商圈热力图")
biz_total = df.groupby("biz_name")["count"].sum().sort_values(ascending=False)
top10_biz = biz_total.head(10).index.tolist()

heatmap_data = np.zeros((len(top10_biz), 24))
for i, biz in enumerate(top10_biz):
    sub = df[df["biz_name"] == biz]
    for hour in range(24):
        hour_data = sub[sub["time_dt"].dt.hour == hour]
        heatmap_data[i, hour] = hour_data["count"].sum()

fig, ax = plt.subplots(figsize=(14, 6))
im = ax.imshow(heatmap_data, aspect="auto", cmap="YlOrRd", interpolation="nearest")
ax.set_xticks(range(24))
```

```
ax.set_xticklabels(range(24), fontsize=10)
ax.set_yticks(range(len(top10_biz)))
ax.set_yticklabels(top10_biz, fontsize=10)
ax.set_xlabel("小时", fontsize=12)
ax.set_ylabel("商圈", fontsize=12)
ax.set_title("Top10商圈每小时人次热力图", fontsize=14, fontweight="bold")
cbar = plt.colorbar(im, ax=ax, label="总人次")
cbar.ax.tick_params(labelsize=10)
# 在每个格子上标注数值（只标注非零且较大的）
for i in range(len(top10_biz)):
    for j in range(24):
        val = heatmap_data[i, j]
        if val > heatmap_data.max() * 0.3:
            text_color = "white" if val > heatmap_data.max() * 0.6 else "black"
            ax.text(j, i, f"{int(val)}", ha="center", va="center",
                    fontsize=7, color=text_color)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_heatmap.png"), dpi=150, bbox_inches="tight")
plt.close()

# 图2：每小时总人次趋势
print("生成图2：每小时总人次趋势")
df["hour"] = df["time_dt"].dt.hour
hourly = df.groupby("hour")["count"].sum()

fig, ax = plt.subplots(figsize=(12, 6))
ax.plot(hourly.index, hourly.values, marker="o", color="#d62728", linewidth=2,
markersize=6)
ax.fill_between(hourly.index, hourly.values, alpha=0.2, color="#d62728")
ax.set_xlabel("小时", fontsize=12)
ax.set_ylabel("总人次", fontsize=12)
ax.set_title("各商圈每小时总人次趋势", fontsize=14, fontweight="bold")
ax.set_xticks(range(24))
ax.grid(alpha=0.3, linestyle="--")
# 标注高峰
peak_hour = hourly.idxmax()
peak_value = hourly.max()
ax.annotate(f"高峰：{peak_hour}时 ({peak_value:.0f}人次)",
```

```
            xy=(peak_hour, peak_value),
            xytext=(peak_hour+2, peak_value*0.9),
            arrowprops=dict(arrowstyle="->", color="red"),
            color="red", fontsize=11, fontweight="bold")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_heatmap_hourly.png"), dpi=150,
bbox_inches="tight")
plt.close()

# 图3：Top5商圈每小时趋势对比
print("生成图3：Top5商圈每小时趋势对比")
top5_biz = biz_total.head(5).index.tolist()
fig, ax = plt.subplots(figsize=(12, 6))
colors = ["#d62728", "#1f77b4", "#2ca02c", "#ff7f0e", "#9467bd"]
for biz, color in zip(top5_biz, colors):
    sub = df[df["biz_name"] == biz]
    biz_hourly = sub.groupby(sub["time_dt"].dt.hour)["count"].sum()
    ax.plot(biz_hourly.index, biz_hourly.values, label=biz, color=color, linewidth=2,
marker="o", markersize=4)
ax.set_xlabel("小时", fontsize=12)
ax.set_ylabel("总人次", fontsize=12)
ax.set_title("Top5商圈每小时人次趋势对比", fontsize=14, fontweight="bold")
ax.set_xticks(range(24))
ax.legend(fontsize=11)
ax.grid(alpha=0.3, linestyle="--")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_heatmap_top5_trend.png"), dpi=150,
bbox_inches="tight")
plt.close()

print(f"\n图片保存在：{FIG_DIR}")
print(f"  fig_heatmap.png")
print(f"  fig_heatmap_hourly.png")
print(f"  fig_heatmap_top5_trend.png")

# 统计摘要
print(f"\n{'='*60}")
print("热力分析统计摘要")
```

```
print(f"{'='*60}")
print(f"总人次: {df['count'].sum()}")
print(f"时间范围: {df['time_dt'].min()} ~ {df['time_dt'].max()}")
print(f"商圈数量: {df['biz_name'].nunique()}")
print(f"\nTop5商圈:")
for biz in top5_biz:
    total = biz_total[biz]
    peak = df[df["biz_name"] == biz].groupby(df["time_dt"].dt.hour)
["count"].sum().idxmax()
    print(f"  {biz}: 总人次{total:.0f}, 高峰{peak}时")
```

5.3 运行结果与解读

生成3张图：商圈热力图、每小时趋势、Top5商圈趋势对比。

5.4 课时小结

- 热力图：Top10商圈×24小时，颜色深浅表示人次；
- 每小时趋势：下午2-4点高峰，凌晨低谷；
- Top5商圈趋势：各商圈高峰时段不同。

课时6：3D轨迹可视化

6.1 课时目标

掌握3D轨迹可视化方法，用经纬度-时间三维展示用户移动轨迹。

6.2 知识点深度讲解

6.2.1 3D轨迹的概念

3D轨迹是在三维空间中展示用户的移动轨迹，三个维度分别是：
1. **X轴**：经度；
2. **Y轴**：纬度；
3. **Z轴**：时间（小时）。

通过3D轨迹可以直观地看到用户在一天内的移动路径和时间分布。

6.2.2 matplotlib 3D绘图

matplotlib的3D绘图需要导入 mpl_toolkits.mplot3d：

```
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot(x, y, z)  # 3D折线
ax.scatter(x, y, z)  # 3D散点
```

6.2.3 视角调整

3D图的视角可以通过 view_init 调整：

```
ax.view_init(elev=30, azim=45)  # elev=仰角, azim=方位角
```

6.3 完整代码（3D轨迹可视化）

```
# ============================================================
# 课时6：3D轨迹可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")

# 读取阶段3结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
df = df.dropna(subset=["time_dt", "lon", "lat"])
df = df.sort_values(["imsi", "time_dt"]).reset_index(drop=True)
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 选择点数最多的前5个用户
top_users =
df.groupby("imsi").size().sort_values(ascending=False).head(5).index.tolist()
print(f"选择Top5用户：{top_users}")

# 图1：3D轨迹图（Top5用户）
print("生成图1：3D轨迹图")
fig = plt.figure(figsize=(14, 10))
ax = fig.add_subplot(111, projection="3d")

colors = ["#d62728", "#1f77b4", "#2ca02c", "#ff7f0e", "#9467bd"]
```

```
for i, (user, color) in enumerate(zip(top_users, colors)):
    user_df = df[df["imsi"] == user].sort_values("time_dt")
    x = user_df["lon"].values
    y = user_df["lat"].values
    z = user_df["time_dt"].dt.hour.values + user_df["time_dt"].dt.minute.values / 60

    # 绘制轨迹线
    ax.plot(x, y, z, color=color, linewidth=1.5, alpha=0.8, label=f"用户{i+1}")
    # 绘制散点（驻留=绿色，出行=红色）
    stay_mask = user_df["label"] == 0
    travel_mask = user_df["label"] == 1
    ax.scatter(x[stay_mask], y[stay_mask], z[stay_mask], c="#2ca02c", s=15, alpha=0.6,
marker="o")
    ax.scatter(x[travel_mask], y[travel_mask], z[travel_mask], c="#d62728", s=15,
alpha=0.6, marker="^")

ax.set_xlabel("经度", fontsize=12, labelpad=10)
ax.set_ylabel("纬度", fontsize=12, labelpad=10)
ax.set_zlabel("时间（小时）", fontsize=12, labelpad=10)
ax.set_title("Top5用户3D移动轨迹（经度-纬度-时间）", fontsize=14, fontweight="bold")
ax.legend(loc="upper left", fontsize=10)
ax.view_init(elev=25, azim=50)
# 添加图例说明
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#2ca02c", markersize=8,
label="驻留"),
    Line2D([0], [0], marker="^", color="w", markerfacecolor="#d62728", markersize=8,
label="出行"),
]
ax.legend(handles=ax.get_legend_handles_labels()[0] + legend_elements,
          loc="upper left", fontsize=9)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_3d_traj.png"), dpi=150, bbox_inches="tight")
plt.close()

# 图2：2D轨迹+时间颜色映射
print("生成图2：2D轨迹+时间颜色映射")
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# 子图1：Top1用户2D轨迹
user1_df = df[df["imsi"] == top_users[0]].sort_values("time_dt")
x1 = user1_df["lon"].values
y1 = user1_df["lat"].values
z1 = user1_df["time_dt"].dt.hour.values + user1_df["time_dt"].dt.minute.values / 60

sc1 = axes[0].scatter(x1, y1, c=z1, cmap="viridis", s=20, alpha=0.7,
edgecolors="black", linewidth=0.5)
axes[0].plot(x1, y1, color="gray", linewidth=0.8, alpha=0.5)
axes[0].set_xlabel("经度", fontsize=12)
axes[0].set_ylabel("纬度", fontsize=12)
axes[0].set_title(f"用户1 2D轨迹（颜色=时间）", fontsize=13, fontweight="bold")
plt.colorbar(sc1, ax=axes[0], label="时间（小时）")
axes[0].grid(alpha=0.3)

# 子图2：所有用户2D轨迹（抽样）
sample_df = df.sample(min(10000, len(df)), random_state=42)
sc2 = axes[1].scatter(sample_df["lon"], sample_df["lat"],
                      c=sample_df["time_dt"].dt.hour, cmap="viridis",
                      s=3, alpha=0.3)
axes[1].set_xlabel("经度", fontsize=12)
axes[1].set_ylabel("纬度", fontsize=12)
axes[1].set_title("所有用户2D轨迹（抽样，颜色=小时）", fontsize=13, fontweight="bold")
plt.colorbar(sc2, ax=axes[1], label="小时")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_2d_traj.png"), dpi=150, bbox_inches="tight")
plt.close()

print(f"\n图片保存在：{FIG_DIR}")
print(f"  fig_3d_traj.png")
print(f"  fig_2d_traj.png")
```

6.4 运行结果与解读

生成2张图：3D轨迹图、2D轨迹+时间颜色映射。

6.5 课时小结

- 3D轨迹：经度-纬度-时间三维展示用户移动路径；
- 2D轨迹+颜色映射：颜色表示时间，直观展示移动规律；
- 驻留=绿色圆点，出行=红色三角。

课时7：综合可视化完整实现（visualize.py）

7.1 课时目标

将所有可视化逻辑整合为完整模块 visualize.py，一次生成6张图。

7.2 完整代码（visualize.py）

```
# ============================================================
# 综合可视化模块：visualize.py
# 功能：一次生成6张分析结果图
# 运行：python visualize.py
# 输入：stage3_label.csv, purpose_result.csv, mode_result.csv, heatmap_result.csv
# 输出：6张PNG图片
# ============================================================

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)


def plot_stage3_pie(df):
    """图1：出行/驻留标签分布饼图。"""
    print("  生成图1：标签分布饼图")
    label_counts = df["label"].value_counts()
    labels = ["驻留" if l == 0 else "出行" for l in label_counts.index]
    colors = ["#2ca02c", "#d62728"]
    fig, ax = plt.subplots(figsize=(8, 6))
    wedges, texts, autotexts = ax.pie(label_counts.values, labels=labels,
                                      autopct="%1.1f%%", colors=colors,
                                      startangle=90, textprops={"fontsize": 14})
    for autotext in autotexts:
        autotext.set_color("white")
        autotext.set_fontweight("bold")
```

```
    ax.set_title("出行/驻留标签分布", fontsize=16, fontweight="bold")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig_stage3_pie.png"), dpi=150,
bbox_inches="tight")
    plt.close()


def plot_hour_line(df):
    """图2：24小时出行人次折线图。"""
    print("  生成图2：24小时折线图")
    df["hour"] = df["time_dt"].dt.hour
    hourly_travel = df[df["label"] == 1].groupby("hour").size().reindex(range(24),
fill_value=0)
    hourly_stay = df[df["label"] == 0].groupby("hour").size().reindex(range(24),
fill_value=0)
    fig, ax = plt.subplots(figsize=(12, 6))
    ax.plot(hourly_travel.index, hourly_travel.values, marker="o", color="#d62728",
            linewidth=2, label="出行", markersize=6)
    ax.plot(hourly_stay.index, hourly_stay.values, marker="s", color="#2ca02c",
            linewidth=2, label="驻留", markersize=6)
    ax.fill_between(hourly_travel.index, hourly_travel.values, alpha=0.2,
color="#d62728")
    ax.set_xlabel("小时", fontsize=12)
    ax.set_ylabel("人次", fontsize=12)
    ax.set_title("24小时出行/驻留人次分布", fontsize=14, fontweight="bold")
    ax.set_xticks(range(24))
    ax.legend(fontsize=12)
    ax.grid(alpha=0.3, linestyle="--")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig_hour_line.png"), dpi=150,
bbox_inches="tight")
    plt.close()


def plot_purpose_bar(purpose_df):
    """图3：出行目的分布柱状图。"""
    print("  生成图3：出行目的柱状图")
    purpose_counts = purpose_df["final_purpose"].value_counts()
```

```
    purpose_order = ["居住", "购物", "旅游", "其他"]
    purpose_counts = purpose_counts.reindex([p for p in purpose_order if p in
purpose_counts.index])
    purpose_colors = ["#1f77b4", "#ff7f0e", "#9467bd", "#7f7f7f"]
    fig, ax = plt.subplots(figsize=(10, 6))
    bars = ax.bar(purpose_counts.index, purpose_counts.values, color=purpose_colors,
                  edgecolor="black", linewidth=1.2)
    ax.set_ylabel("驻留段数", fontsize=12)
    ax.set_title("出行目的分布", fontsize=14, fontweight="bold")
    for bar, val in zip(bars, purpose_counts.values):
        ax.text(bar.get_x() + bar.get_width()/2, val + 5,
                f"{val}\n({val/len(purpose_df)*100:.1f}%)",
                ha="center", va="bottom", fontsize=11, fontweight="bold")
    ax.set_ylim(0, purpose_counts.max() * 1.2)
    ax.grid(axis="y", alpha=0.3, linestyle="--")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig_purpose_bar.png"), dpi=150,
bbox_inches="tight")
    plt.close()


def plot_mode_bar(mode_df):
    """图4：出行方式分布柱状图。"""
    print("  生成图4：出行方式柱状图")
    mode_counts = mode_df["final_mode"].value_counts()
    mode_order = ["地铁", "公交", "小汽车", "其他"]
    mode_counts = mode_counts.reindex([m for m in mode_order if m in
mode_counts.index])
    mode_colors = ["#d62728", "#1f77b4", "#ff7f0e", "#7f7f7f"]
    fig, ax = plt.subplots(figsize=(10, 6))
    bars = ax.bar(mode_counts.index, mode_counts.values, color=mode_colors,
                  edgecolor="black", linewidth=1.2)
    ax.set_ylabel("出行段数", fontsize=12)
    ax.set_title("出行方式分布", fontsize=14, fontweight="bold")
    for bar, val in zip(bars, mode_counts.values):
        ax.text(bar.get_x() + bar.get_width()/2, val + 1,
                f"{val}\n({val/len(mode_df)*100:.1f}%)",
                ha="center", va="bottom", fontsize=11, fontweight="bold")
```

```
    ax.set_ylim(0, mode_counts.max() * 1.2)
    ax.grid(axis="y", alpha=0.3, linestyle="--")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig_mode_bar.png"), dpi=150,
bbox_inches="tight")
    plt.close()


def plot_heatmap(heatmap_df):
    """图5：商圈热力图。"""
    print("  生成图5：商圈热力图")
    heatmap_df["time_dt"] = pd.to_datetime(heatmap_df["time"])
    biz_total = heatmap_df.groupby("biz_name")
["count"].sum().sort_values(ascending=False)
    top10_biz = biz_total.head(10).index.tolist()
    heatmap_data = np.zeros((len(top10_biz), 24))
    for i, biz in enumerate(top10_biz):
        sub = heatmap_df[heatmap_df["biz_name"] == biz]
        for hour in range(24):
            heatmap_data[i, hour] = sub[sub["time_dt"].dt.hour == hour]["count"].sum()
    fig, ax = plt.subplots(figsize=(14, 6))
    im = ax.imshow(heatmap_data, aspect="auto", cmap="YlOrRd", interpolation="nearest")
    ax.set_xticks(range(24))
    ax.set_xticklabels(range(24), fontsize=10)
    ax.set_yticks(range(len(top10_biz)))
    ax.set_yticklabels(top10_biz, fontsize=10)
    ax.set_xlabel("小时", fontsize=12)
    ax.set_ylabel("商圈", fontsize=12)
    ax.set_title("Top10商圈每小时人次热力图", fontsize=14, fontweight="bold")
    plt.colorbar(im, ax=ax, label="总人次")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig_heatmap.png"), dpi=150, bbox_inches="tight")
    plt.close()


def plot_3d_traj(df):
    """图6：用户3D轨迹图。"""
    print("  生成图6：3D轨迹图")
```

```
    top_users =
df.groupby("imsi").size().sort_values(ascending=False).head(5).index.tolist()
    fig = plt.figure(figsize=(14, 10))
    ax = fig.add_subplot(111, projection="3d")
    colors = ["#d62728", "#1f77b4", "#2ca02c", "#ff7f0e", "#9467bd"]
    for i, (user, color) in enumerate(zip(top_users, colors)):
        user_df = df[df["imsi"] == user].sort_values("time_dt")
        x = user_df["lon"].values
        y = user_df["lat"].values
        z = user_df["time_dt"].dt.hour.values + user_df["time_dt"].dt.minute.values /
60
        ax.plot(x, y, z, color=color, linewidth=1.5, alpha=0.8, label=f"用户{i+1}")
    ax.set_xlabel("经度", fontsize=12, labelpad=10)
    ax.set_ylabel("纬度", fontsize=12, labelpad=10)
    ax.set_zlabel("时间（小时）", fontsize=12, labelpad=10)
    ax.set_title("Top5用户3D移动轨迹", fontsize=14, fontweight="bold")
    ax.legend(loc="upper left", fontsize=10)
    ax.view_init(elev=25, azim=50)
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig_3d_traj.png"), dpi=150, bbox_inches="tight")
    plt.close()


def visualize_all():
    """执行所有可视化。"""
    print("=" * 60)
    print("综合可视化")
    print("=" * 60)

    # 读取数据
    print("\n读取数据...")
    df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                     dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
    df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")
    df = df.dropna(subset=["time_dt", "lon", "lat"])
    print(f"  阶段3结果: {len(df)} 行")

    purpose_df = pd.read_csv(os.path.join(RESULT_DIR, "purpose_result.csv"),
```

```
                                dtype={"imsi": str, "start_time": str, "end_time": str})
    print(f"  出行目的: {len(purpose_df)} 段")

    mode_df = pd.read_csv(os.path.join(RESULT_DIR, "mode_result.csv"),
                          dtype={"imsi": str, "start_time": str, "end_time": str})
    print(f"  出行方式: {len(mode_df)} 段")

    heatmap_df = pd.read_csv(os.path.join(RESULT_DIR, "heatmap_result.csv"))
    print(f"  热力结果: {len(heatmap_df)} 行")

    # 生成图片
    print(f"\n生成图片...")
    plot_stage3_pie(df)
    plot_hour_line(df)
    plot_purpose_bar(purpose_df)
    plot_mode_bar(mode_df)
    plot_heatmap(heatmap_df)
    plot_3d_traj(df)

    print(f"\n所有图片已保存到: {FIG_DIR}")
    print(f"  fig_stage3_pie.png")
    print(f"  fig_hour_line.png")
    print(f"  fig_purpose_bar.png")
    print(f"  fig_mode_bar.png")
    print(f"  fig_heatmap.png")
    print(f"  fig_3d_traj.png")

    print("\n" + "=" * 60)
    print("综合可视化完成！")
    print("=" * 60)


if __name__ == "__main__":
    visualize_all()
```

7.3 运行结果与解读

运行后一次生成6张图。

7.4 课时小结

- visualize.py 整合所有可视化逻辑；
- 6张图：标签饼图、小时折线图、目的柱状图、方式柱状图、热力图、3D轨迹图；
- 一次运行生成所有图片。

课时8：可视化优化与答辩准备

8.1 课时目标

掌握可视化优化技巧，准备项目答辩材料。

8.2 知识点深度讲解

8.2.1 可视化优化技巧

1. **分辨率优化**：savefig dpi=150或300，保证清晰度；
2. **布局优化**：tight_layout自动调整子图间距；
3. **颜色优化**：使用色盲友好配色（如viridis、plasma）；
4. **字体优化**：统一字体大小，标题加粗；
5. **标注优化**：关键数据点添加数值标注；
6. **图例优化**：图例位置合理，不遮挡数据。

8.2.2 答辩准备

项目答辩需要准备：
1. **项目背景**：交通时空大数据分析的意义和应用场景；
2. **数据介绍**：数据集来源、规模、字段；
3. **技术栈**：Python + numpy + pandas + matplotlib；
4. **算法流程**：数据清洗→出行驻留识别→出行目的→出行方式→实时热力→可视化；
5. **核心算法**：三阶段出行驻留识别、k-means聚类、滑动窗口热力；
6. **结果展示**：6张可视化图，关键数据指标；
7. **创新点**：Haversine精确距离、多指标选K、向量化性能优化；
8. **总结与展望**：项目成果、不足、未来改进方向。

8.3 完整代码（可视化优化与答辩准备）

```
# ============================================================
# 课时8：可视化优化与答辩准备
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取所有结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
purpose_df = pd.read_csv(os.path.join(RESULT_DIR, "purpose_result.csv"),
                         dtype={"imsi": str, "start_time": str, "end_time": str})
mode_df = pd.read_csv(os.path.join(RESULT_DIR, "mode_result.csv"),
                      dtype={"imsi": str, "start_time": str, "end_time": str})
heatmap_df = pd.read_csv(os.path.join(RESULT_DIR, "heatmap_result.csv"))

# 答辩关键数据
print("=" * 60)
print("项目答辩关键数据")
print("=" * 60)

print(f"\n1. 数据规模:")
print(f"  原始数据: 约53万行")
print(f"  清洗后: {len(df)} 行, {df['imsi'].nunique()} 用户")
print(f"  实时流数据: 约25万行")
print(f"  基站数据: 约5万行")
print(f"  商业区数据: 104个商圈")

print(f"\n2. 出行驻留识别:")
print(f"  驻留点: {(df['label']==0).sum()} ({(df['label']==0).mean()*100:.1f}%)")
print(f"  出行点: {(df['label']==1).sum()} ({(df['label']==1).mean()*100:.1f}%)")
```

```
print(f"\n3. 出行目的识别:")
print(f"  驻留段: {len(purpose_df)}")
for purpose in ["居住", "购物", "旅游", "其他"]:
    count = (purpose_df["final_purpose"] == purpose).sum()
    print(f"  {purpose}: {count} ({count/len(purpose_df)*100:.1f}%)")

print(f"\n4. 出行方式识别:")
print(f"  出行段: {len(mode_df)}")
for mode in ["地铁", "公交", "小汽车", "其他"]:
    count = (mode_df["final_mode"] == mode).sum()
    print(f"  {mode}: {count} ({count/len(mode_df)*100:.1f}%)")

print(f"\n5. 实时热力分析:")
print(f"  总人次: {heatmap_df['count'].sum()}")
print(f"  时间点数: {heatmap_df['time'].nunique()}")
print(f"  商圈数量: {heatmap_df['biz_name'].nunique()}")
biz_total = heatmap_df.groupby("biz_name")["count"].sum().sort_values(ascending=False)
print(f"  Top5商圈:")
for biz in biz_total.head(5).index:
    print(f"    {biz}: {biz_total[biz]:.0f}人次")

print(f"\n6. 技术栈:")
print(f"  Python 3.13")
print(f"  NumPy 2.5（向量化计算）")
print(f"  Pandas 3.0（数据处理）")
print(f"  Matplotlib 3.11（可视化）")

print(f"\n7. 核心算法:")
print(f"  数据清洗: 9条规则（字段抽取/去脏imsi/时间转换/基站关联/去重）")
print(f"  出行驻留: 三阶段（低频点过滤→k-means聚类→规则精判）")
print(f"  出行目的: 时间规则+空间匹配（<1km）")
print(f"  出行方式: 速度规则+空间匹配（<500m，地铁≥2点/公交≥3点）")
print(f"  实时热力: 滑动窗口（每分钟过去10分钟）")

print(f"\n8. 创新点:")
print(f"  Haversine球面距离（精确计算，非欧氏距离近似）")
print(f"  多指标选K（SSE+轮廓系数+邓恩指数）")
print(f"  向量化性能优化（批量空间匹配，比逐行循环快10倍以上）")
```

```
print(f"  预计算+滑动累加（热力统计性能优化）")

print(f"\n9. 可视化成果:")
print(f"  fig_stage3_pie.png - 出行/驻留标签分布")
print(f"  fig_hour_line.png   - 24小时出行人次趋势")
print(f"  fig_purpose_bar.png - 出行目的分布")
print(f"  fig_mode_bar.png    - 出行方式分布")
print(f"  fig_heatmap.png     - 商圈热力图")
print(f"  fig_3d_traj.png     - 用户3D轨迹")

print(f"\n10. 总结与展望:")
print(f"  成果：完整实现交通时空大数据分析系统，6大模块，6张可视化图")
print(f"  不足：基站定位精度有限（200-500m），影响空间匹配精度")
print(f"  展望：引入GPS数据提升精度，增加实时流处理，构建Web Dashboard")
```

8.4 运行结果与解读

输出答辩关键数据，涵盖数据规模、算法、结果、创新点、可视化成果。

8.5 课时小结

- 可视化优化：分辨率、布局、颜色、字体、标注、图例；
- 答辩准备：10个方面的关键数据和说明；
- 项目成果：6大模块，6张可视化图，完整可运行。

三、本日小结

1. **综合可视化总览**：6张图（标签饼图、目的柱状图、方式柱状图、小时折线图、热力图、3D轨迹图）；
2. **matplotlib高级技巧**：子图、gridspec布局、rcParams样式、annotate注释、twinx双Y轴、colorbar；
3. **出行驻留可视化**：标签饼图（驻留87%/出行13%）、24小时折线图（早/晚高峰）、空间分布；
4. **出行目的与方式可视化**：柱状图+饼图，标注数量和比例；
5. **实时热力可视化**：热力图（Top10商圈×24小时）、每小时趋势、Top5商圈对比；
6. **3D轨迹可视化**：经度-纬度-时间三维展示，2D轨迹+时间颜色映射；
7. **visualize.py**：整合所有可视化，一次生成6张图；

8. **答辩准备**：10个方面的关键数据和说明。

四、本日作业

1. 运行 visualize.py，确认6张图全部生成；
2. 查看6张图，写一段综合分析；
3. 准备项目答辩PPT，包含项目背景、算法流程、结果展示、创新点；
4. 预习 Day11：思考"如何把所有模块整合为一个完整的main.py流程？"